

# 자연어처리1 4주차 과제

|                  |                                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------|
| 🔍 과목             | 자연어 처리1 1주차, 자연어 처리1 2주차, 자연어처리1 3주차, 자연어처리1 4주차, 자연어처리1 4주차 과제                                     |
| ➤ 큰 계획           |  <u>02. 자연어처리1</u> |
| 🔍 공부             | 02. 자연어처리1                                                                                          |
| ☰ 카테고리           |                                                                                                     |
| ✨ 공부 완료<br>정도    | 0회독                                                                                                 |
| Σ yesterday      | <input type="checkbox"/>                                                                            |
| ➤ Parent<br>item | <u>자연어처리1 4주차</u>                                                                                   |

## 자연어처리 4주차 실습 과제

1. PTB 데이터셋 평가 결과 캡처 및 분석(2점)
  - ch02/count\_method\_big.py 수행 결과 캡처 (1점)
  - ch02/count\_method\_big.py - 분석 (1점)
2. 3장(word2vec, p133) create\_contexts\_target() 함수 분석 (1점)

### ch02/count\_method\_big.py 분석

1. target에 corpus의 양쪽 window size 만큼 제외한 배열을 대입한다.
2. idx가 target이 가능한 범위를 차례로 증가하면서
3. 각 idx의 양쪽 window size 만큼의 배열( $t=0 \rightarrow$  target 제외)을 context 배열에 추가한다.
4. idx가 target 가능 범위( $\text{len}(\text{corpus}) - \text{window\_size} - 1$ ) 끝까지 2,3 반복
5. context, target 반환

### count\_method\_big

```

window_size = 2
wordvec_size = 100

corpus, word_to_id, id_to_word = ptb.load_data('train')
vocab_size = len(word_to_id)
print('동시발생 수 계산 ...')
C = create_co_matrix(corpus, vocab_size, window_size)
print('PPMI 계산 ...')
W = ppmi(C, verbose=True)

print('calculating SVD ...')
try:
    # truncated SVD (빠르다!)
    from sklearn.utils.extmath import randomized_svd
    U, S, V = randomized_svd(W, n_components=wordvec_size, n_iter=5,
                             random_state=None)
except ImportError:
    # SVD (느리다)
    U, S, V = np.linalg.svd(W)

word_vecs = U[:, :wordvec_size]

querys = ['you', 'year', 'car', 'toyota']
for query in querys:
    most_similar(query, word_to_id, id_to_word, word_vecs, top=5)

```

## 실행 예시

```

100 98.0% 완료
101 99.0% 완료
102 100.0% 완료
103 calculating SVD ...
104
105 [query] you
106 i: 0.7483155727386475
107 we: 0.5815799236297607
108 anybody: 0.5485869646072388
109 'll: 0.5000213384628296
110 've: 0.49942976236343384
111
112 [query] year
113 month: 0.6898108124732971
114 next: 0.6699495911598206
115 quarter: 0.6553088426589966
116 last: 0.6183187365531921
117 june: 0.6079219579696655
118
119 [query] car
120 luxury: 0.6823889017105103
121 auto: 0.6690361499786377
122 vehicle: 0.5497623682022095
123 cars: 0.543609082698822
124 corsica: 0.5101791620254517
125
126 [query] toyota
127 motor: 0.7344875335693359
128 nissan: 0.6999912858009338
129 motors: 0.6830120086669922
130 honda: 0.6472179889678955
131 lexus: 0.6196972131729126
132

```

## 1. C에 동시발생 행렬 만들어 대입

### create\_co\_matrix

```

def create_co_matrix(corpus, vocab_size, window_size=1):
    '''동시발생 행렬 생성

    :param corpus: 말뭉치(단어 ID 목록)
    :param vocab_size: 어휘 수
    :param window_size: 윈도우 크기(윈도우 크기가 1이면 타깃 단어 좌우 한 단어씩)
    :return: 동시발생 행렬
    '''
    corpus_size = len(corpus)
    co_matrix = np.zeros((vocab_size, vocab_size), dtype=np.int32)

    for idx, word_id in enumerate(corpus):
        for i in range(1, window_size + 1):

```

```

left_idx = idx - i
right_idx = idx + i

if left_idx >= 0:
    left_word_id = corpus[left_idx]
    co_matrix[word_id, left_word_id] += 1

if right_idx < corpus_size:
    right_word_id = corpus[right_idx]
    co_matrix[word_id, right_word_id] += 1

return co_matrix

```

1-1. vocab 수 x vocab 수 행렬 생성

1-2.  $\{ \text{idx} \mid 0 \leq \text{idx} \leq \text{corpus.size}()-1 \}$ , left idx는 idx 기준 왼쪽 window index, right idx는 idx 기준 오른쪽 window index

1-3. left idx의 corpus id를 left\_word\_id에 저장 및 빈도 수 갱신, right idx도 같은 알고리즘.

1-4. 동시발생 행렬 return

2. 만든 동시발생 행렬을 매개변수로 한 ppmi 함수 호출

## ppmi

```

def ppmi(C, verbose=False, eps = 1e-8):
    """PPMI(점별 상호정보량) 생성

    :param C: 동시발생 행렬
    :param verbose: 진행 상황을 출력할지 여부
    :return:
    """
    M = np.zeros_like(C, dtype=np.float32)
    N = np.sum(C)
    S = np.sum(C, axis=0)
    total = C.shape[0] * C.shape[1]
    cnt = 0

    for i in range(C.shape[0]):

```

```

for j in range(C.shape[1]):
    pmi = np.log2(C[i, j] * N / (S[j]*S[i]) + eps)
    M[i, j] = max(0, pmi)

    if verbose:
        cnt += 1
        if cnt % (total//100) == 0:
            print('%.1f%% 완료' % (100*cnt/total))
return M

```

2-1. M은 C와 동일한 shape과 값은 0인 행렬, N은 C행렬 값들의 총합, S는 C의 열방향 합들을 저장한 각 어휘의 총 빈도 수들, total은 C의 원소 총 개수.

2-2. i는 행, j는 열을 순서대로 index를 의미하고 C의 행렬에서 pmi 식을 계산해서 M에 대입(양수 혹은 0).

3. W를 SVD 연산하여 U, S, V 형태로 나눈다.

4. word\_vecs에 U의 모든 행과 열은 0부터 wordvec\_size-1만큼 대입

5. querys 안에 있는 단어들과 가장 유사도가 높은 단어들 5개를 출력.

## ch03/create\_contexts\_target() 함수 분석

```

def create_contexts_target(corpus, window_size=1):
    '''맥락과 타깃 생성

    :param corpus: 말뭉치(단어 ID 목록)
    :param window_size: 윈도우 크기(윈도우 크기가 1이면 타깃 단어 좌우 한 단어씩
    이 맥락에 포함)
    :return:
    '''
    target = corpus[window_size:-window_size]
    contexts = []

    for idx in range(window_size, len(corpus)-window_size):
        cs = []
        for t in range(-window_size, window_size + 1):
            if t == 0:

```

```

        continue
        cs.append(corpus[idx + t])
        contexts.append(cs)

    return np.array(contexts), np.array(target)

```

corpus : 말뭉치

window\_size : context(맥락)에 포함되는 단어의 범위(숫자)를 뜻함

target : context(맥락)을 통해 추측하고자 하는 중앙의 단어  
즉 context를 입력으로 하여 타겟의 출현확률을 높이하고자 함

1. target에 corpus의 양쪽 window\_size만큼 제외한 배열을 대입한다.  
→ corpus>window\_size:-window\_size]를 통해 타겟 단어들만 추출한다.
2. idx가 target이 가능한 범위 (window\_size부터 len(corpus) - window\_size - 1까지)를 차례로 증가하면서
3. 각 idx 위치에서 양쪽 window\_size만큼 떨어진 단어들을 모아 context 배열에 추가한다.  
→ 이때 t == 0인 경우 (즉, 중심 단어 target 자체)는 제외하고 양옆 단어만 추가한다.
4. idx가 target 가능한 범위 (len(corpus) - window\_size - 1) 끝까지 도달할 때까지 2, 3을 반복한다.
5. contexts, target 배열을 numpy 배열로 변환하여 반환한다.

## 결과 예시

- corpus = [1, 2, 3, 4, 5]
- window\_size = 1

인 경우에 return 값은 아래와 같다

- target = [2, 3, 4]
- contexts = [[1, 3], [2, 4], [3, 5]]

## 실행 예시

```

import numpy as np
def create_contexts_target(corpus, window_size=1):
    '''맥락과 타겟 생성

    :param corpus: 말뭉치(단어 ID 목록)
    :param window_size: 윈도우 크기(윈도우 크기가 1이면 타겟 단어 좌우 한 단어씩이 맥락에 포함)
    :return:
    '''
    target = corpus[window_size:-window_size]
    contexts = []

    for idx in range(window_size, len(corpus)-window_size):
        cs = []
        for t in range(-window_size, window_size + 1):
            if t == 0:
                continue
            cs.append(corpus[idx + t])
        contexts.append(cs)

    return np.array(contexts), np.array(target)

```

✓ 0.5s

Python

```

corpus = [1, 2, 3, 4, 5]
window_size = 1
context, target= create_contexts_target(corpus, window_size=window_size)

print(context)

```

✓ 0.0s

Python

```

[[1 3]
 [2 4]
 [3 5]]

```